Riverbend — Not Built (omissions, next priorities, scaling risks)

Scoped against the **actual** implementation (verified in code), not the README's aspirations.

A. Deliberately not built (MVP boundaries)

- **Customer self-service accounts / portal.** Customers interact only via the public form and email; there

is no customer login. (Staff/admin auth IS built — Better Auth, Feature 010.)

- **Verified-channel automatic cancellation.** Cancellation intent is detected and routed to staff review

(`cancellation_requires_verification`, D23); the system never auto-cancels or infers order ownership from an email address. No OTP / verified-link cancel flow.

- **Semantic/business-level duplicate detection.** Idempotency is an explicit `submission_id` key

(`UNIQUE(shop_id, source, submission_id)`), not content similarity across different operations.

- **Hard financial-grade budget reservations.** Budget is a LiteLLM-native **bounded soft cap**, not a

two-phase transactional reservation (constitution 2.0.0).

- **Staff invitation workflow / advanced RBAC.** Admin creates staff directly; roles are a small fixed set

(PLATFORM_ADMIN / SHOP_ADMIN / SHOP_STAFF). No invitations, SSO, JWT, or fine-grained permissions.

- **Multi-shop selector / credential-replace UI** in the portal (minor polish; the authorization supports it).
- **Observability stack** (OpenTelemetry/Phoenix tracing, dashboards, alerts) and a data-retention/TTL sweeper.
- **Operations:** HA/multi-region, autoscaling the worker on queue depth, a managed-Redis path (there is no

Redis — pg-boss is Postgres-backed by design).

- **Resend production domain** (SPF/DKIM/MX) — email runs against a Resend-managed receiving address for the

MVP; arbitrary-recipient delivery needs a verified sending domain. New-inbound routing is built (D25: `shops.inbound_address` → shop), but the per-shop receiving addresses must be real verified addresses

in production; new-inbound de-dup relies on a stable RFC Message-ID.

- **Rich email normalization.** New-inbound bodies use the plain-text part with a best-effort HTML→text

fallback; **attachments are skipped** (not parsed/stored), and DMARC/SPF results are fetched but not yet gated on. Deep MIME/multipart edge cases are not handled beyond what Resend pre-splits.

- **Fine-grained failure visibility.** Distinct states exist (`held_budget` , `invalid_output:…` ,

`needs_review`), but transient causes **collapse at the DLQ boundary** — a timeout, a 503, and malformed JSON all surface to staff as `processing attempts exhausted` . The staff portal also does not yet show the retry-attempt count or remaining budget (only spend). (A genuine `429` is now retried, not parked — D25; and `held_budget` requests are now re-driven automatically once the cap clears — D25.)

B. What I'd build next (priority order)

1. **Verified-channel cancellation** (replace the manual review for the common case). *Why:* it's the most

frequent human-review reason; a signed one-click "confirm cancel" link closes the loop safely. *Trigger:* cancellation volume in `needs_review` becomes a staff burden.

1. **A dedicated restricted DB role for Better Auth** (it currently uses the migrator/owner role). *Why:*

least privilege for the identity store. *Trigger:* before any real production deploy.

1. **Worker autoscaling on queue depth + per-shop concurrency fairness.** *Why:* a single busy shop can

starve others under load. *Trigger:* p95 job latency rises or the pg-boss backlog grows.

1. **Observability** (OTel traces around the model call + budget, a few RED/USE dashboards, spend alerts).

Why: you can't operate spend/latency you can't see. *Trigger:* first real traffic.

1. **Retention/TTL sweeper** for `email_events` /old requests. *Why:* unbounded ingestion tables. *Trigger:*

table growth.

C. What breaks first at 100 shops (ranked; evidence-based)

> "100 shops" is a hypothesis for load testing; the ranking is by code evidence, not measured limits.

1. **Postgres connection + pool pressure (TOP risk).** *Evidence:* every tenant request opens a `withTenant`

client and `set_config`; the api, worker, Better Auth, and pg-boss all hold pools against one Postgres; Cloud SQL caps connections. *Signal:* connection-wait time, `too many clients`, p95 query latency. *Fix:* a pooler (PgBouncer) / tuned pool sizes / read replicas; batch the reconciler.

1. **LLM rate limits & spend.** *Evidence:* extraction is one model call per request through one LiteLLM

proxy; OpenAI is ~RPM/TPM limited and the budget cap is a soft cap. *Signal:* 429s from the proxy, budget `held_budget` states, rising cost/day. *Fix:* per-shop queue shaping, backoff, raise/stagger budgets, cheaper model for easy cases.

1. **Worker throughput / queue backlog.** *Evidence:* the worker is a small fixed instance count (Terraform

`worker_min_instances`, "no queue-depth autoscaling" is a noted gap) and the reconciler + email poller run on one interval loop; one slow shop's jobs share the lane. *Signal:* pg-boss active/queued depth, job age. *Fix:* autoscale workers, shard queues per shop, move the 60s email poll to webhooks at volume.

Secondary: email webhook/poll throughput (polling is 60s and single-flight — fine for MVP, add webhooks at scale); manual shop onboarding (provisioning is a script); no monitoring/incident tooling yet; migrations run at api startup (fine for one instance, needs a gated migration step with many instances).